

Application of Agentic Workflows to Enable Rich Interactive Fiction Worlds

Gene Foxwell

Woolf University / Udacity School of Artificial Intelligence

Design of Autonomous and Semi-Autonomous Agentic Workflows

Overview

This project looks at creating an Agentic Workflow for enabling users to interact with an Interactive Fiction environment such as those crafted by Infocom¹ in 1980's. We approach this problem in two phases. In phase 1, we take a story and compile it into a Graph Database. In this phase the Agentic Workflow is autonomous with the Agents focusing on Named Entity Recognition and the relationships between them. In phase 2, we allow a player to interact with the story, generating a new personalized version of the story on the fly based on their input.

We will cover this system as follows – first we'll describe the Architecture and Design, next we'll discuss the system Logic and various behaviors. After this, we'll discuss safety issues that may arise from the use of this system, behaviors observed from using the system, ethical concerns, and future work.

System Architecture and Design

As mentioned in the Overview, we split our system into two primary phases. In both phases we use a Graph Database – specifically mgraph-db² - as our underlying datastore. We choose to use a Graph Database rather than a traditional RDS style database for two reasons:

1. Data regarding fictional stories is highly connected – friends have friends, have locations etc. Graphs allow us to efficiently model these connections.
2. GraphRAG (Retrieval Augmented Generation where the data source is a Graph Database) has shown considerable advantages when used to help Agent's reason about relationships (Han, H 2025).

¹ [Infocom - Wikipedia](#)

² [MGraph-DB](#)

3. Graph Representations have been shown to be useful in creating Interactive Fiction environments in other related work (Ammanabrolu 2020)

MGraph was chosen as it allows us to build a lightweight system where the graph remains primarily in memory – reducing the burden of additional third-party tools and setup that would be needed for more traditional graph databases like Neo4j or AWS Neptune.

We built all our agents using the Pydantic AI³ library. This library handled API calls to the underlying LLM, tool use, and retry limits. As it is already handled by the library, we do not explicitly define the tools in our system prompts.

In phase 1 of the system, we build an Agentic Workflow whose primary task is to extract the information we need to construct our initial Graph Database. We build this database in stages:

1. An agent to recognize and extract Named Entities from the story. We focus on primary characters and the objects they interact with.
2. An agent extracts all important locations from the story.
3. We use another agent to document the relationships between the various characters and objects. For example, a character might have a “is_friends” relationship with another. We allow the LLM to determine what the relationship labels should be based on the story rather than impose a specific ontology.
4. We build a “map” of the locations by extracting all the linked locations from the story using another Agent.

The agents in steps 3 & 4 above provide their results as a collection of “triplets”. These triplets are of the form:

³ [Pydantic AI | Pydantic Docs](#)

subject $\rightarrow$ *predicate* $\rightarrow$ *object*

Here the predicate represents a relationship between the subject and the object. For example, Johnny is friends with Sarah could be modelled with Johnny as the subject, Sarah as the object, and is friends with as the predicate.

A separate, non-Agentic subroutine is used to translate the triplets into the knowledge graph. As extracting all this information is both time-consuming and relatively expensive (high-token usage), once the database is populated it is exported to a json file so we can reload it later.

Figure 1 provides a visual representation of the process in action.

PHASE 1 - GRAPH GENERATION

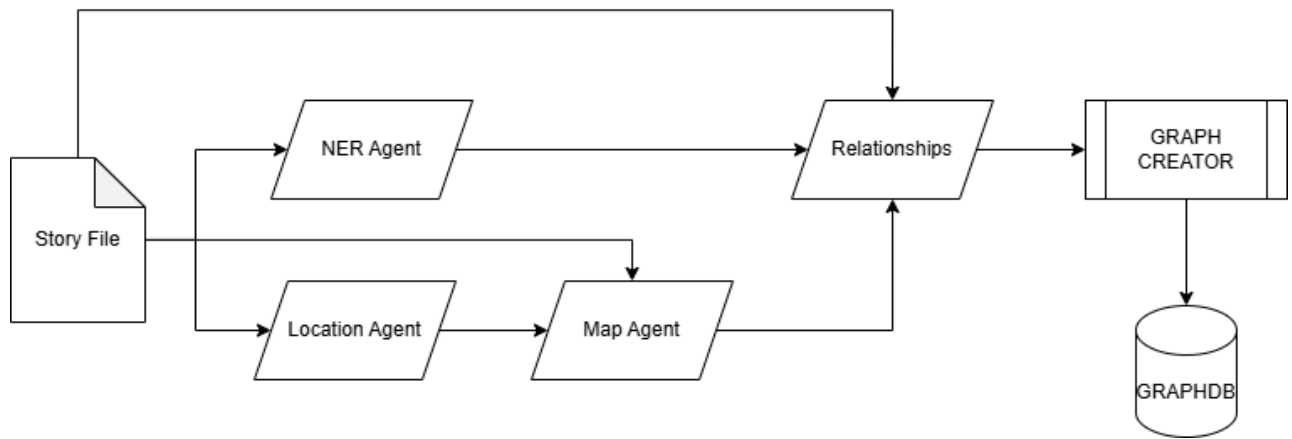


Figure 1 Graph Generation

In phase 2, we allow the user to interact with the system. Providing a model for how we could simulate a user's interaction with a fictional world provided some challenges. We looked at (Yuan 2025) as an inspirational starting point for this challenge. For the primary interaction loop, we broke the problem down into two primary agents:

- **Avatar Agents:** Avatar Agents represent a specific character in the story. We create a separate Avatar Agent for each major character represented in the underlying Graph Database. Each Avatar Agents task is to consider the most recent story developments and generate a set of actions that reflect how that character would (or perhaps would not) respond. The Agent then updates the underlying Graph Database using tools (see table 1) and returns a natural language description of those actions.
- **Storyteller Agent:** The job of the storyteller is to take the collective actions of the Avatar Agents, the current data in the Graph Database, the user's input, and generate the next phase of the story from the perspective of the character the player has chosen to masquerade as. The storyteller has access to tools that provide it with read-only access to the Graph Database.

To support the two Agents above, we built several support Agents:

- **Guard Rail Agent:** This agent analyzes the users' input to determine if it meets safety and ethical standards. If the input does not pass the required standards, the input is rejected by the system, and the user is asked to provide a different input.
- **Router Agent:** This agent is responsible for determining if a user's input is a request for information about the story, or an attempt to move the story forward.

Informational requests are routed to the information agent. Story requests are routed to Avatar Agents and the Storyteller Agent.
- **Informational Agent:** This agent answers questions about the current state of the story. For example, if the user asks who Zania's friend is, that request would be routed to this agent.

- **Summary Agent:** This agent provides an overall summary of the story based on the underlying Graph Database. It is available as a menu option in the systems user interface.

Figure 2 provides a visualization of how the user's request is handled throughout the process.

The Storyteller and Avatar Agents each track a simple history of their previous interactions that is inserted into the context at the start of each turn. This history consists of the last N request / response pairs where the request is the user's input, and the response is the agent's eventually response.

Tools	Purpose
Query By Name	Returns any triplets from the Graph Database where either the Subject or Object name matches the provided name.
Query By Type	Returns any triplets from the Graph Database where either the Subject or Object type matches the provided type.
Insert Predicate	Inserts a new predicate between a given Subject and Object.
Remove Predicate	Removes a predicate between a given Subject and Object.

The table below shows the system prompts used for each agent in the system:

Agent	System Prompt	Tools	Notes
Character Extraction Agent	You are an expert in identifying characters in a story. You will be given a story. Your task is to: * identify all primary characters in the story. * generate a description of the character from story.		Returns a collection of "PlotEntities" – each has a name, description, and type fields

Location Extraction Agent	You are an expert in identifying locations in a story. You will be given a story. Your task is to: * identify all primary locations in the story. * generate a description of the location based on the story.		Returns a collection of "PlotEntities" – each has a name, description, and type fields
Map Creation Agent	You are an expert at mapping out the connections between locations from a story. You will be given a story. You will be given a list of locations from that story. Your task is to: * Construct a collection of triplets where each triplet represents a connection between two locations in the story. * Assign the "connects_to" label as the predicate for each triplet.		Returns a collection of GraphTriplets which are triplets of type Object, Predicate, Subject.
Character Relationship Agent	You are an expert in mapping out the relationships between characters in a story. You are also an expert in mapping out the relationships between characters and objects in a story. You will be given a story.		Returns a collection of GraphTriplets which are triplets of type Object, Predicate, Subject.

	You will be given a list of characters and objects from that story. You will be given a list of locations from that story. Your task is to: * construct a collection of subject, predicate, object triplets representing the important relationships in the story.		
Avatar Agent	You are an expert in determining changes to the story caused a players declared actions. The character the player controls is named: {character_name} You will be given a users input. You will be given tools to query the current Graph Database that represents the story world. You will be given tools to add/remove relationships from that Graph Database. Your task is to: * Find any changes to the world that would be caused by the players actions. * Update the knowledge graph as needed. * Generate any dialogue this character needs to move the story forward.	Query By Entity Name, Query By Entity Type, Insert Predicate, Remove Predicate	

	* Summarize the actions you took in a user friendly manner.		
Storyteller Agent	You are an expert at writing stories from the perspective of a given character. You will be given the name of the character. You will be given a summary of the each character in the stories next moves. You will be given the previous responses from the story teller. You will be given tools to query the underlying graph database representing the story for more information. Your task is to: * Synthesize the actions of each character in the story in a coherent story output, told from the perspective of the given story character. * Output should be in "story" format.	Query By Entity Name, Query By Entity Type	
Informational Agent	You are an expert at answering questions about the state of a story. You will be given a graph database representing the current story. You will be given the most recent history of the story so far.	Query By Entity Name, Query By Entity Type	

	You will be given tools to query the graph database. You will be given a user query. Your task is to: * Use the graph database to answer the users query. * Provide evidence from the graph database to support your answer.		
Routing Agent	You are an expert in differentiating between questions about a story and a users attempt to add to or move the story forward. Your task is to: * Analyze if the user is asking for information about the story or game world. * Return "QUERY" if the user making a query, "STORY" otherwise.		
State Summarization Agent	You are an expert at summarizing the current state of a story from the perspective of a specific character. You will be given a character name. You will be given tools to query the knowledge graph for information about that character and their relationships to other entities in the story. Your task is to:	Query By Entity Name	

	* Summarize the current state of the story from the perspective of the given character, including the character's current situation, relationships, and any relevant events or locations, based on the information available in the knowledge graph. * Your summary should be concise and focus on the most important aspects of the character's perspective		
Guard Rails Agent	You are a guardian agent, responsible for making sure that inputs and outputs from the storyteller agent are appropriate and do not contain any harmful or inappropriate content. You will review the story updates generated by the story teller agent and ensure that they adhere to ethical guidelines and do not contain any content that could be considered harmful, offensive, or inappropriate. If you find any content that violates these guidelines, you will flag it and prevent it from being presented to the user, and respond with "CANNOT UPDATE".		

	Inappropriate Content includes, but is not limited to: * Hate speech or discriminatory language * Explicit or graphic content that is not suitable for all audiences * Content that promotes violence or self-harm * Any content that could be considered offensive or harmful to individuals or groups based on factors such as race, religion, or sexual orientation. * Any content that violates the terms of service of the platform on which this system is being used. * Any content that violates legal or ethical standards for content in the relevant jurisdiction.		
--	--	--	--

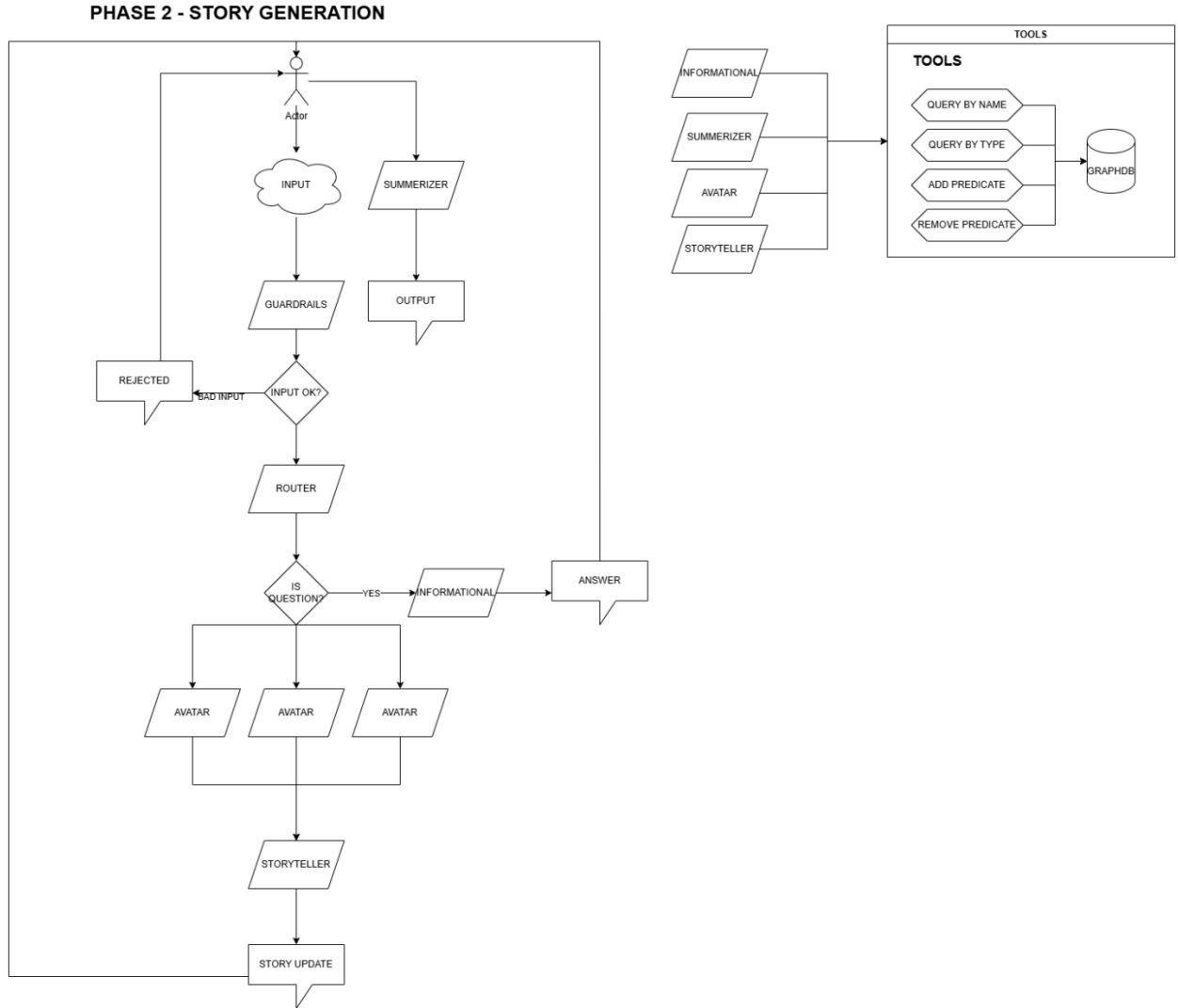


Figure 2 Story Generation

Decision Logic and Behavior

The overall behavior of the system breaks down as follows:

1. **Graph Construction:** Before “phase 2” can be executed, the Graph Database needs to be populated. This is done autonomously with the Agentic Workflow reading the data, extracting the appropriate entities and relationships, and passing them into static subroutine.

2. **Entry Point:** The entry point into user interaction is a simple input prompt. If the user enters exit, the system ends the program. An input of summary calls upon the Summarization Agent to provide a summary of the state so far. Otherwise, the users input is passed on to the Guard Rails agent.
3. **Guard Rails:** As discussed in the previous section, this agent works as a classifier. If it returns CANNOT UPDATE, the user's input is ignored. Otherwise, the users input is passed along to the Routing Agent.
4. **Routing:** Here it is decided if the user is asking a question about the story (for example, who is Samuel?) or is adding to the story themselves. (Zania calls Samuel for help). Questions are routed to the Informational Agent. Other requests are sent to the Avatar / Storytelling Agents.
5. **Avatar Agents:** Each character in the story is associated with an Avatar Agent. The Avatar Agents determine the decisions/actions each character takes to advance the story and update the underlying Graph Database accordingly. The outputs from all Avatar agents are then concatenated and sent to the Storyteller Agent.
6. **Storytelling:** The Storyteller agent combines the decisions / outputs of the Avatar Agents, the user's input, and the current data in the Graph Database to create a new update to the story. It then generates this update from the perspective of the player's character. This is then printed onto the console, and the player is prompted for their next input.
7. **Information:** Information requests are routed to the Informational Agent. This agent performs RAG to answer the players questions about the current state of the Graph Database / Story world.

As alluded to in the previous section, Avatar Agents and the Storyteller agent implement a simple memory system. This system is implemented using a deque. The deque holds the last N interactions providing continuity in the both the Agents behavior and the story overall.

We use Pydantic's built in tool retry functionality to handle failure states. In the event the system is simply unable to proceed further, we exit the user input loop.

Safety, Reliability, Transparency

To improve user safety, we implemented a Guard Rails agent (as described in the System Architecture and Design section). This agent was used as a classifier to determine if the user's input was intended to guide the system into harmful or unethical behaviors not intended by the designer. When such input is detected, the user is informed that the request violates the system's ethical and safety guidelines and asks the user to provide a separate input.

To provide insight into what the system is doing, both the stage of processing (which character is being updated) and the current tool call are output to the terminal. This allows the user to see what search terms and updates are being run against the Graph Database. In addition, at the start of "turn", the system displays a summary of all actions it took on their characters behalf before outputting the next segment of the story.

To help combat hallucinations, we split the task amongst multiple different types of Agents. This allowed each agent to focus on a single task, equipped only with the tools, memory, and input relevant to that task. The idea was that this should help cross contamination of concepts. Graph Databases were also chosen to provide more reliable results as they have

already been noted to provide state-of-the-art results in multi-hop reasoning tasks, as well as seeing success in similar tasks to ones explored here.

Observed Behavior and Limitations

The story updates generated by the system can be quite verbose, delving deep into the questions the main character is asking themselves. It is not clear at this time if this is simply the result of the stories used to test the system itself, or if it is an intrinsic property of this system. Determining this would require a new set of experiments outside the scope of this work.

This system does appear to hallucinate some relationships or concepts that were not originally intended by the author of the story. Given the goal of the system is to provide a personalized story experience, this is not something we would look to fix – in our case some level of hallucination should be considered a feature rather than a bug.

It can also interpret the users' input in unexpected ways. While testing the Guard Rails system the experimenter suggested that the main character remove their clothing. The expected result was that the Guard Rail would block this interaction. Due to a defect (now repaired) it got to the storyteller system, where this request was translated into a metaphor for the main character showing vulnerability instead.

Unfortunately, the system sometimes has trouble determining the correct search terms for using the tools provided. In these cases, the Agents will hit the max retries limit on tool use in Pydantic AI, causing a python exception to be thrown.

Additionally, the story structure that gets created isn't always self-consistent. Some of this can be attributed to non-self-consistent inputs from the human operator, other consistency errors were the result of the system hallucinating plot details on its own.

We are also limited by the size of the story file we can process. Currently, we do not chunk the story file, which means our story as well as any supplemental information must fit into the agent's context window. If the story is too large, we risk the agent losing part of the story and creating an incomplete or inconsistent Graph Database to represent it.

Ethical and Responsible Use Considerations

This system was designed to tell interactive stories – which leaves it open to abuse by both end users and story providers alike. It is intended for entertainment purposes only. From a story provider point of view, this system should not be promoted as a replacement for human connection. It is also not intended to be used to tell stories that contain content that is illegal or unethical in nature. While the Guardian Agent included as part of this system can mitigate some of these issues, it is not perfect and cannot be 100% relied upon to protect the user – especially if the seed story already violates ethical guidelines.

Additionally, while this system does generate stories via the LLM, these stories are intended to be for personal consumption, not for commercial use. Not only are LLM's trained on other author's source material (making copyright and ethical concerns regarding publishing / monetization murky) but in many cases the end user may not directly own the rights to the plot used to generate the initial knowledge graph.

Future Improvements

The system presented is a basic prototype of the underlying concept. Further refinement would include:

- Providing a more robust Ontology for the initial Knowledge Graph creation. This would allow for more reliable initialization of the Graph Database.
- A proper web or application-based interface to allow users to properly interact with the system.
- Integration with an Image Generator to allow users to visualize various situations.
- Integration with simple game engines, for example RPGMaker or ren'py to allow developers to bring potential NPC's to life more easily.
- Optimization. The current system is quite unoptimized and takes some time to generate each new response. Using thread safe methods to update the Graph Database would allow us to split the Avatar agents into their threads, speeding up the time between turns considerably.

References

Ammanabrolu, P., Cheung, W., Tu, D., Broniec, W., & Riedl, M. O. (2020). Bringing stories alive: Generating interactive fiction worlds. *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*.

<https://arxiv.org/pdf/2001.10161>

Han, H., Ma, L., Shomer, H., Wang, Y., Lei, Y., Guo, K., Hua, Z., Long, B., Liu, H., Aggarwal, C. C., & Tang, J. (2025). *RAG vs. GraphRAG: A systematic evaluation and key insights*. arXiv. <https://doi.org/10.48550/arXiv.2502.11371>

Yuan, X., Yuan, S., & colleagues. (2025). BookWorld: From novels to interactive agent societies for creative story generation. *arXiv*. <https://arxiv.org/html/2504.14538v1>